PAULT DOCUMENTATION

2026 02 19 Close All Gaps Design

Source: docs/plans/2026-02-19-close-all-gaps-design.md

> **Theme:** Clean Technical

Generated: February 26, 2026

Close All Gaps — Design Document

Date: 2026-02-19

Status: Implemented

Plan file: `.claude/plans/async-honking-dolphin.md`

Goal

Close five gaps that were blocking Pault's v1.0 readiness:

1. **API Key Settings UI** — users had no way to enter API keys; `KeychainService` existed but had no UI

2. **AIAssistPanel Stub Tabs** — Variables, Tags, and Score tabs showed "coming soon"; `AIService` backends were already implemented

3. **Silent AI Errors** — `catch` blocks dropped errors without user feedback

4. **Appearance Preferences** — font size and compact mode were disabled with "coming in a future update" captions

5. **Hotkey Key Recorder** — global hotkey was hardcoded to Cmd+Shift+P with no way to change it

---

Architecture

1. API Key Settings UI

A new **"AI" tab** in `PreferencesView` containing `AISettingsTab` (private struct).

Three provider sections: Claude, OpenAI, Ollama (Local).

Each section provides:

- `SecureField` for API key (Claude, OpenAI only)
- `TextField` for model name
- Ollama has a base URL field instead of API key
- "Test Connection" button with inline result indicator

Storage:

- API keys → `KeychainService(key: "ai.apikey.\(provider)")` — exactly matches the lookup key in `AIService.buildRequest`
- Model overrides → `@AppStorage("ai.model.\(provider)")`
- Ollama base URL → `@AppStorage("ai.baseURL.ollama")`

Test connection pattern: Races `AIService.shared.improve("Hello", config:)` against a 5-second `Task.sleep` via `withThrowingTaskGroup`. Result shown as inline ■ / ■ label.

State: `enum TestResult: Equatable { case testing, ok, failed(String) }` — cleaner than a boolean + optional message.

Keys are loaded from Keychain on `.onAppear` (not `@AppStorage` — secrets don't belong in `UserDefaults`).

2. AIAssistPanel Stub Tabs

Three new private structs added to `AIAssistPanel.swift`:

`VariablesTabContent`

- "Suggest Variables" → `AIService.suggestVariables(prompt:config:)` → `[VariableSuggestion]`
- Each suggestion row shows placeholder + description with "Insert" button
- Insert strips existing `{{ }}` wrapping and re-appends `{{token}}` to `prompt.content`
- "Insert All" button bulk-inserts all suggestions

`TagsTabContent`

- "Suggest Tags" → `AIService.autoTag(prompt:config:)` → `[String]` (up to 3)
- Tag pills rendered as `Buttons`; already-attached pills are disabled
- Tap: looks up existing `Tag` by name (case-insensitive) via `@Query`, or creates new one, then appends to `prompt.tags`
- Uses `@Environment(\.modelContext)` for tag creation

`ScoreTabContent`

- "Analyse" → `AIService.qualityScore(prompt:config:)` → `QualityScore`
- Four `ProgressView(value:total:)` rows (Clarity, Specificity, Completeness, Conciseness)
- Overall score shown with `.font(.title3)`

3. Error Surfacing

`AIErrorBar` — new shared component in `AIAssistPanel.swift`:

- Red triangle icon + message text + dismiss × button
- `Color.red.opacity(0.06)` background, `RoundedRectangle(cornerRadius: 4)` clip
- Used in all five AI panels (Improve, Variables, Tags, Score, and pre-existing ResponsePanel/RefinementLoopView)

All `catch` blocks now set an `error: String?` state variable and show `AIErrorBar` when non-nil. The existing `ResponsePanel` and `RefinementLoopView` already surfaced errors correctly — no changes needed there.

4. Appearance Preferences

Font size wired to `RichTextEditor`:

- `@AppStorage("fontSizePreference")` read inside `RichTextEditor` (NSViewRepresentable struct)
- `editorFontSize: CGFloat` computed property: small=13, medium=15, large=17
- Applied in `makeNSView` and re-applied in `updateNSView` via `context.coordinator.textView?.font`
- `updateNSView` fires whenever `@AppStorage` changes, so switching preference updates live

Compact mode wired to `SidebarView`:

- `@AppStorage("useCompactMode")` in `SidebarView`

- `.listRowInsets(EdgeInsets(top: compact ? 4 : 8, leading: 8, bottom: compact ? 4 : 8, trailing: 8))` on each `ForEach` row

Both controls had `.disabled(true)` and "coming in a future update" captions removed from `AppearanceTab`.

5. Hotkey Key Recorder

`KeyRecorderView` — `NSViewRepresentable` wrapping `KeyRecorderNSView: NSView`:

`KeyRecorderNSView` behaviour:

- `mouseDown` → enters recording mode, becomes first responder, draws highlighted border
- `keyDown` → captures `event.keyCode` (Carbon virtual key code) + converts `NSEvent.ModifierFlags` to Carbon modifier mask → calls `onRecorded` closure → exits recording mode
- `Escape` → cancels recording without change
- `resignFirstResponder` → exits recording mode if focus is lost
- Custom `draw(_:)` renders a rounded rect with label or "Recording…" state

Carbon modifier conversion:

```
if nsFlags.contains(.command) { mods |= UInt32(cmdKey) }
if nsFlags.contains(.shift) { mods |= UInt32(shiftKey) }
// ... etc.
```

Display string built from modifier glyphs (Ctrl+Option+Shift+Cmd+) + key character from a Carbon key code → glyph map.

`PreferencesView.hotkeyTab` replaces the static text display with `KeyRecorderView`. On record:

1. Writes `hotkeyKeyCode` and `hotkeyModifiers` to `UserDefaults`

2. Updates `globalHotkey` display string via `@AppStorage`

3. Calls `GlobalHotkeyManager.shared.register(keyCode:modifiers:)` immediately — hotkey is live without restart

A "Reset to Default (Cmd+Shift+P)" button restores the original binding.

`AppDelegate.setupGlobalHotkey` now reads from `UserDefaults` with fallback to defaults (`0x23` / `cmdKey|shiftKey`) using an `Int.nonZero` helper that returns nil for un-set (zero) values.

`Notification.Name.toggleLauncher` added — re-registration from Preferences posts this notification; `AppDelegate` observes it to call `toggleLauncher()`. This ensures the notification-based path and the direct-call path both trigger the launcher correctly.

Key Design Decisions

Why `TestResult` is `Equatable` instead of a separate boolean + optional?

SwiftUI switch/case pattern in `testConnectionRow` requires == comparisons. The enum is the natural fit for a 3-state result.

Why load API keys in `.onAppear` rather than via `@AppStorage`?

`UserDefaults` (the backing store for `@AppStorage`) is world-readable within the app sandbox and can be extracted from `~/Library/Preferences`. Keychain is the correct store for credentials.

Why re-apply font in `updateNSView` unconditionally?

`NSFont.systemFont(ofSize:)` is cheap, and the conditional check would need to compare the font object — less readable for negligible gain.

Why `Int.nonZero` instead of a default value in UserDefaults registration?

Registering defaults via `UserDefaults.standard.register` runs at launch, but Carbon's key code `0` is a valid key (A). Using `nonZero` nil-coalesces only on the truly absent case.

Files Created / Modified

| File | Status | Task |

|---|---|---|

| `Pault/PreferencesView.swift` | Modified (AI tab, Appearance enabled, hotkey recorder) | 1, 4, 5 |

| `Pault/AIAssistPanel.swift` | Modified (Variables, Tags, Score tabs + AIErrorBar) | 2, 3 |

| `Pault/KeyRecorderView.swift` | Created | 5 |

| `Pault/PaultApp.swift` | Modified (`.toggleLauncher` notification) | 5 |

| `Pault/AppDelegate.swift` | Modified (UserDefaults hotkey read, notification observer) | 5 |

| `Pault/RichTextEditor.swift` | Modified (font size AppStorage) | 4 |

| `Pault/SidebarView.swift` | Modified (compact mode list row insets) | 4 |